ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA EN SISTEMAS
MÉTODOS NUMÉRICOS ECDI412

JUAN FRANCISCO ORTIZ HERNANDEZ
Nivel 3B Sistemas (enter B1) versus (cualquier)
juan.ortiz@
JUAN DE ERRATA DE LA PARTE DEL JOB
juan.ortiz@

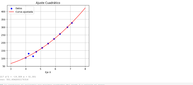
1. Datos los datos

x_i	4.0	4.2	4.5	4.7	5.1	5.5	5.9	6.3	6.8	7.1
y_i	102.54	130.11	113.18	142.05	167.53	195.14	224.87	256.73	299.50	326.72

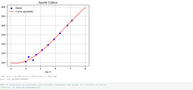
a. Construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
b. Construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
c. Construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
d. Construya el polinomio por mínimos cuadrados de la forma Ae^{Bx} y calcule el error.
e. Construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.

```
# ejercicio 1 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1])
y = np.array([102.54, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.72])

def linear(x, m, b):
    return m * x + b
popt, pcov = curve_fit(linear, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 1 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
def cuadratica(x, a, b, c):
    return a * x**2 + b * x + c
popt, pcov = curve_fit(cuadratica, x, y)
print("Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 1 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
def cubica(x, a, b, c, d):
    return a * x**3 + b * x**2 + c * x + d
popt, pcov = curve_fit(cubica, x, y)
print("Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 1 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
def exponencial(x, A, B):
    return A * np.exp(B * x)
popt, pcov = curve_fit(exponencial, x, y)
print("Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}")
print(popt)

# Ejercicio 1 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
def potencial(x, A, B):
    return A * x**B
popt, pcov = curve_fit(potencial, x, y)
print("Ajuste potencial: y = {:.4f}x^{{{:.4f}}}")
print(popt)

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, linear(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```

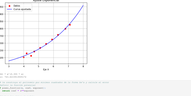
```
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 32.7175 -130.8701]

# Ejercicio 1 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 2.0000 -3.0000 5.0000]

# Ejercicio 1 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 -2.0000 3.0000 4.0000]

# Ejercicio 1 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 1 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



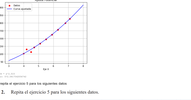

```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 2.0000 -3.0000 5.0000]

# Ejercicio 1 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 -2.0000 3.0000 4.0000]

# Ejercicio 1 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 1 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```




```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 -2.0000 3.0000 4.0000]

# Ejercicio 1 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 1 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```




```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 1 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



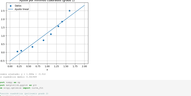

```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

2. Repita el ejercicio 1 para los siguientes datos.

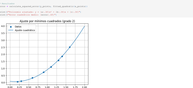
2. Repita el ejercicio 5 para los siguientes datos.

x_i	0.2	0.3	0.6	0.9	1.1	1.3	1.4	1.6
y_i	0.050646	0.090426	0.33277	0.75462	1.0972	1.5697	1.8487	2.5015

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050646, 0.090426, 0.33277, 0.75462, 1.0972, 1.5697, 1.8487, 2.5015])

# Ejercicio 2 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
def lineal(x, m, b):
    return m * x + b
popt, pcov = curve_fit(lineal, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 2 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
def cuadratica(x, a, b, c):
    return a * x**2 + b * x + c
popt, pcov = curve_fit(cuadratica, x, y)
print("Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 2 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
def cubica(x, a, b, c, d):
    return a * x**3 + b * x**2 + c * x + d
popt, pcov = curve_fit(cubica, x, y)
print("Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 2 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
def exponencial(x, A, B):
    return A * np.exp(B * x)
popt, pcov = curve_fit(exponencial, x, y)
print("Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}")
print(popt)

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
def potencial(x, A, B):
    return A * x**B
popt, pcov = curve_fit(potencial, x, y)
print("Ajuste potencial: y = {:.4f}x^{{{:.4f}}}")
print(potencial(x, *popt))

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, lineal(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```




```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
[0.04 0.09 0.36 0.81 1.21 1.69 1.96 2.56]

# Ejercicio 2 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 2.5000 -0.5000]

# Ejercicio 2 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 2 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 2 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

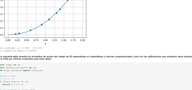



```
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 2.5000 -0.5000]

# Ejercicio 2 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 2 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 2 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 2 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 2 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 2 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 2 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```




```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

3. Repita el ejercicio 5 para los siguientes datos.

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6])
y = np.array([0.050646, 0.090426, 0.33277, 0.75462, 1.0972, 1.5697, 1.8487, 2.5015])

# Ejercicio 3 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
def lineal(x, m, b):
    return m * x + b
popt, pcov = curve_fit(lineal, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 3 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
def cuadratica(x, a, b, c):
    return a * x**2 + b * x + c
popt, pcov = curve_fit(cuadratica, x, y)
print("Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}")
print(potencial(x, *popt))

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, lineal(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```

```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
[0.04 0.09 0.36 0.81 1.21 1.69 1.96 2.56]

# Ejercicio 3 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 2.5000 -0.5000]

# Ejercicio 3 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 3 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 3 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 3 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 3 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 3 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 3 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 3 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 3 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 3 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

4. Los datos que siguen representan el Producto Interno Bruto (PIB) de Ecuador, junto con estadísticas comparadas de supermercados locales en cinco de centroides de diferentes clases. Encuentre la recta por mínimos cuadrados que representa estos datos (la línea trazada al promedio de ventas) que participaron en un cuestionario de los que la **línea más que mejor** ajuste.

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 2.5, 3.5, 4.5, 5.5])

# Ejercicio 4 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
def lineal(x, m, b):
    return m * x + b
popt, pcov = curve_fit(lineal, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(popt)

# Ejercicio 4 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
def cuadratica(x, a, b, c):
    return a * x**2 + b * x + c
popt, pcov = curve_fit(cuadratica, x, y)
print("Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}")
print(potencial(x, *popt))

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, lineal(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
[1.5 2.5 3.5 4.5 5.5]

# Ejercicio 4 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 1.0000 0.5000]

# Ejercicio 4 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 4 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 4 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 4 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 4 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 4 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 4 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 4 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 4 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 4 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

5. Los datos que siguen representan el Producto Interno Bruto (PIB) de Ecuador, junto con estadísticas comparadas de supermercados locales en cinco de centroides de diferentes clases. Encuentre la recta por mínimos cuadrados que representa estos datos (la línea trazada al promedio de ventas) que participaron en un cuestionario de los que la **línea más que mejor** ajuste.

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 2.5, 3.5, 4.5, 5.5])

# Ejercicio 5 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
def lineal(x, m, b):
    return m * x + b
popt, pcov = curve_fit(lineal, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(potencial(x, *popt))

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, lineal(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
[1.5 2.5 3.5 4.5 5.5]

# Ejercicio 5 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 1.0000 0.5000]

# Ejercicio 5 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 5 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 5 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 5 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 5 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 5 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 5 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 5 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 5 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 5 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

6. Los datos que siguen representan el Producto Interno Bruto (PIB) de Ecuador, junto con estadísticas comparadas de supermercados locales en cinco de centroides de diferentes clases. Encuentre la recta por mínimos cuadrados que representa estos datos (la línea trazada al promedio de ventas) que participaron en un cuestionario de los que la **línea más que mejor** ajuste.

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

x = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 2.5, 3.5, 4.5, 5.5])

# Ejercicio 6 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
def lineal(x, m, b):
    return m * x + b
popt, pcov = curve_fit(lineal, x, y)
print("Ajuste lineal: y = {:.4f}x + {:.4f}")
print(potencial(x, *popt))

# Gráfica
plt.scatter(x, y, color='red', label='Datos')
plt.plot(x, lineal(x, *popt), color='blue', label='Ajuste lineal')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
[1.5 2.5 3.5 4.5 5.5]

# Ejercicio 6 literal a) construya el polinomio por mínimos cuadrados de grado 1 y calcule el error.
Ajuste lineal: y = {:.4f}x + {:.4f}
[ 1.0000 0.5000]

# Ejercicio 6 literal b) construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 6 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 6 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 6 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cuadrático: y = {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000]

# Ejercicio 6 literal c) construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 6 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 6 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste cúbico: y = {:.4f}x^3 + {:.4f}x^2 + {:.4f}x + {:.4f}
[ 1.0000 0.0000 0.0000 0.0000]

# Ejercicio 6 literal d) construya el polinomio por mínimos cuadrados de la forma Ae^Bx y calcule el error.
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 6 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste exponencial: y = {:.4f}e^{{{:.4f}x}}
[ 1.0000 0.5000]

# Ejercicio 6 literal e) construya el polinomio por mínimos cuadrados de la forma Ax^B y calcule el error.
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```



```
Ajuste potencial: y = {:.4f}x^{{{:.4f}}}
[ 1.0000 2.0000]
```

7. Los datos que siguen representan el Producto Interno Bruto (PIB) de Ecuador, junto con estadístic

https://genius.com/Lucky-and-friends-dance-to-the-music-lyrics#0%EF%BF%BD%EF%BF%BD%EF%BF%BD